

Evaluating Agricultural Loans and Expenditure Allocation in Nigeria Using Double Machine Learning

William Hankins¹, Alexis Villacis², Ani Katchova³, Ivette Contreras⁴

¹The Ohio State University. Email: hankins.181@osu.edu

²The Ohio State University. Email: villacis.9@osu.edu

³The Ohio State University. Email: katchova.1@osu.edu

⁴ The World Bank. Email: icontreras@worldbank.org

Contents

1	Reproducibility Package README	2
2	Data Availability and Provenance Statements	2
2.1	Statement about Rights	2
2.2	Summary of Availability	2
3	Getting Started	3
3.1	Outline and Folder Structure	3
3.2	How to Reproduce	3
4	Dataset List	5
5	List of Tables and Programs	5
6	Computational Requirements	6
6.1	Software Requirements	6
6.2	Installing Python Dependencies	7
6.3	Key Configuration Parameters	8
7	References	8

Reproducibility Package README

The code within this replication package executes the analysis detailed in the paper “Agricultural Loans and Expenditure Allocation in Nigeria: Evidence from Double Machine Learning” using Stata, R, and Python. This package includes all code required to go from the raw survey microdata to the replication of all tables and figures in the analysis. The Stata master file (`00_master.do`) manages sequential execution of all data construction code. Python estimation is run in a second stage, these programs are computationally taxing and can take between minutes to over a day to run depending on specifications and hardware. Our primary objective is to ensure accessibility and credibility in replicating our research methodology and results.

Data Availability and Provenance Statements

The analysis uses two rounds of Nigeria’s General Household Survey – Panel (GHS-Panel), a nationally representative panel survey conducted by the National Bureau of Statistics (NBS) in collaboration with the World Bank. The paper draws on Wave 4 (2018/19) and Wave 5 (2023/24). Raw GHS-Panel microdata are publicly available from the World Bank Microdata Library (Wave 4: <https://microdata.worldbank.org/index.php/catalog/3557> and Wave 5: <https://microdata.worldbank.org/index.php/catalog/6410>); free registration is required.

In addition, the analysis uses pre-processed LSMS panel data compiled by the Evans School Policy Analysis and Research Group (EPAR), available at:

[https://github.com/EvansSchoolPolicyAnalysisAndResearch/LSMS-Data-Dissemination/
tree/main/Nigeria%20GHS](https://github.com/EvansSchoolPolicyAnalysisAndResearch/LSMS-Data-Dissemination/tree/main/Nigeria%20GHS)

Statement about Rights

I certify that the author(s) of the manuscript have legitimate access to and permission to use the data used in this manuscript.

Summary of Availability

- ☒ All data are publicly available.
- ☐ Some data cannot be made publicly available.
- ☐ No data can be made publicly available.

Both the World Bank GHS-Panel microdata and the EPAR processed files are freely and publicly accessible at the links above. Place the Wave 4 World Bank files in `Source Data/Nigeria GHS Wave 4/Raw DTA files/` and the Wave 5 files in `Source Data/Nigeria GHS Wave 5/Raw DTA files/`. Place the EPAR processed household files in the corresponding `Source Data/Nigeria GHS Wave 4/epar_data/` and `Source Data/Nigeria GHS Wave 5/epar_data/` folders before running any code.

Getting Started

Outline and Folder Structure

The reproducibility package consists of five top-level folders:

- **Source Data/** — GHS survey files (download from the links above).
- **Stata Code/** — All Stata do-files for data cleaning and table production. Final analysis dataset is written to `Stata Code/Stata Data Landing/DML Cleaned Data.dta`.
- **R Code/** — Single script (`FIES Index Calculation.r`) for Rasch-model food insecurity index; called automatically by the Stata master.
- **Python Code/** — DML estimation, sensitivity analysis, and figure scripts.
- **Tables and Figures/** — DML logs, nuisance caches, tables, and final figure outputs.

```
$ROOT/
|-- 00_master.do           # Stata data and table entry point
|-- 00_figures.py         # Descriptive and appendix figure runner
|-- 01_figures.py         # Main figure runner
|-- requirements.txt       # Python package dependencies
|-- Source Data/
|   |-- Nigeria GHS Wave 4/
|   |   |-- Raw DTA files/
|   |   '--- epar_data/
|   '--- Nigeria GHS Wave 5/
|       |-- Raw DTA files/
|       '--- epar_data/
|-- Stata Code/
|   |-- Stata Data Collection and Cleaning Scripts/
|   '--- Stata Data Landing/DML Cleaned Data.dta
|-- R Code/FIES Index Calculation.r
|-- Python Code/
|   |-- Python DML Scripts/
|   '--- Pthon Figures Scripts/
'--- Tables and Figures/   # Logs, caches, tables, and figures
```

How to Reproduce

PART 1: Stata — Data Construction. Open `00_master.do` from the project root and run it without modification. The script constructs paths automatically from the current working directory. If an error is returned, comments inside should guide troubleshooting, but there are two primary issues that may arise:

1. If `00_master.do` returns an error that a Stata `.dta` file could not be found then either: 1) the data was not stored properly as outlined in [Summary of Availability](#), or 2) the path macros are not working correctly. In the latter case, `global root "`c(pwd)'"` should set the root to the current reproduction folder. If needed, set it manually following the comments near the top of `00_master.do`.

2. If an error occurs because the Rscript does not run properly or because FIES dataset (`household_probs_long.csv`) cannot be found for Subsection 2.2 of `00_master.do`, then the `do` file is not correctly executing the Rscript. If Rscript is not on your system `PATH`, set the executable path manually as described in the comments near the top of `00_master.do`.

This installs required Stata packages, calls the R FIES script, and produces `Stata Code/Stata Data Landing/DML Cleaned Data.dta` and supporting Stata inputs used by the downstream figures. Subsequent Python stages read these outputs. Estimated runtime: ≤ 1 min.

PART 2: Python — Install Dependencies. Install all required packages with a single command (see Section 6.2 for the full `requirements.txt`):

```
pip install -r requirements.txt
```

PART 3: Python — Estimation and Output. Run the following scripts in order. No path changes are needed; each script resolves paths relative to its own file location.

- **PART 3.1 — Descriptive figures and summary statistics.** `00_figures.py` (run from `$ROOT`)
Produces descriptive and appendix figures, including Figures A1–A5, and summary statistics. Outputs are saved to `Tables` and `Figures/`.
- **PART 3.2 — Standard DML results.** Python Code/Python DML Scripts/
`DML Identification.py`
Set `tipping_point = 0` and select the standard outcomes required for the tables and figures, including fertilizer utilization when reproducing its figure. Paper results use `N_REP = 30` and `k_fold_vector = [5]`. Logs and nuisance predictions are written to `Tables` and `Figures/` and `Tables` and `Figures/nuisance_cache/`.
- **PART 3.3 — Balanced tipping-point analysis for Figure 4.** Run `DML Identification.py` again with `tipping_point = 1`, `HTE = True`, both core outcomes selected, `k_fold_vector = [5]`, and `N_REP = 30`. This uses the common two-wave farm/nonfarm sample and writes `Tables` and `Figures/nuisance_cache/`
`tipping_common/tipping_joint_K5_R30.npz`.
- **PART 3.4 — Omitted Variable Bias Benchmarking.** Python Code/Python DML Scripts/
`DML Benchmarking.py`
Computes omitted-variable bounds by re-estimating with each control dropped in turn. Output CSV files are written to `Tables` and `Figures/`.
- **PART 3.5 — Main figures.** `01_figures.py` (run from `$ROOT`)
After the standard and tipping outputs exist, this runs Figures 2–6 and the sensitivity forest plot. Figure 4 reads the balanced tipping output from PART 3.3.

Cache behavior. Nuisance predictions are cached under `Tables` and `Figures/nuisance_cache/`. Subsequent identical runs load the cache automatically when `re_estimate_nuisance=False`. The tipping calculation does not use cached predictions. Set `re_estimate_nuisance=True` after changing the source data, folds, repetition count, controls, learners, or outcomes.

Dataset List

Dataset	Description	Provided
Source Data/Nigeria GHS Wave 4/Raw DTA files/*.dta	Raw GHS Wave 4 (2018/19) survey modules. (link above)	No (public)
Source Data/Nigeria GHS Wave 5/Raw DTA files/*.dta	Raw GHS Wave 5 (2023/24) survey modules. (link above)	No (public)
Source Data/Nigeria GHS Wave 4/epar_data/*.dta	EPAR LSMS processed data Wave 4 (linked above).	No (public)
Source Data/Nigeria GHS Wave 5/epar_data/*.dta	EPAR LSMS processed data Wave 5 (linked above).	No (public)

List of Tables and Programs

Table / Figure	Program	Output file
Balance Table	Stata Code/ Stata Data Collection and Cleaning Scripts/ Stata Output Tables Scripts/ Balance Table.do	Tables and Figures/ balance.table.tex
Main DML Results (ATE, HTE, GATE)	Python Code/ Python DML Scripts/ DML Identification.py	Tables and Figures/ DML Identification_v*_*.txt
Balanced tipping-point results	Python Code/ Python DML Scripts/ DML Identification.py tipping_point = 1	with Tables and Figures/ nuisance.cache/ tipping.common/ tipping_joint_K5_R30.npz; versioned log under Tables and Figures/ tipping.common/ Tables and Figures/ benchmark*.csv
Sensitivity Bounds	Python Code/ Python DML Scripts/ DML Benchmarking.py	Tables and Figures/ benchmark*.csv
Summary Statistics	00_figures.py	Tables and Figures/ quartile.summary*.csv
Figure A1 (FIES KDE)	Python Code/ Python Figures Scripts/ DML Figure A1.py	Tables and Figures/ Figure_A1.pdf

Table / Figure	Program	Output file
Figure A2 (Farm-expenditure KDE by treatment)	Python Code/ Pthon Figures Scripts/ DML Figure A2.py	Tables and Figures/ Figure A2.pdf
Figure A3 (Farm-size KDE by treatment)	Python Code/ Pthon Figures Scripts/ DML Figure A3.py	Tables and Figures/ Figure A3.pdf
Figure A4 (Farm-size KDE by loan formality)	Python Code/ Pthon Figures Scripts/ DML Figure A4.py	Tables and Figures/ Figure A4.pdf
Figure A5 (Farm expenditures by farm size)	Python Code/ Pthon Figures Scripts/ DML Figure A5.py	Tables and Figures/ Figure_farm_expense_ by_size.pdf
Figure 4 (balanced tipping point)	Python Code/ Pthon Figures Scripts/ DML Figure 4.py	Tables and Figures/ tipping_common/ Figure_tipping_point_ common_sample.pdf
Main figure suite	01_figures.py	Figures 2–6 and sensitivity forest plot
Sensitivity Forest Plot	Python Code/ Pthon Figures Scripts/ DML Figure Benchmark.py	Tables and Figures/ sensitivity_forest_plot_ K5_Rep30.pdf

Computational Requirements

Software Requirements

Stata

Stata/MP or Stata/SE version 17 or later. The following user-written packages are *automatically installed* by `00_master.do` on the first run:

Package	Source	Purpose
<code>univar</code>	SSC	Univariate statistics
<code>ietoolkit</code>	SSC	IE data management utilities
<code>winsor2</code>	SSC	Winsorizing continuous variables
<code>reghdfe</code>	SSC	High-dimensional fixed-effects estimation

R

R version 4.0 or later. Called via shell from `00_master.do` for the FIES index computation. R must be on the system PATH or its path must be set in the shell command in `00_master.do`. Required package: `RM.weights` (install via `install.packages("RM.weights")`).

Python

Python 3.11 or later (3.12 used for paper results). Table 3 lists all required packages with the exact versions used.

Table 3: Required Python packages

Package	Version	Purpose
numpy	2.0.0	Numerical arrays
pandas	2.2.2	Data manipulation; reads <code>.dta</code> via <code>read_stata</code>
scikit-learn	1.7.2	Ensemble learners and cross-validation
doubleml	0.11.1	DML Partial Linear Regression
statsmodels	0.14.6	Clustered OLS for GATE estimation
matplotlib	3.9.0	All figures
scipy	1.16.2	Kernel density estimation
joblib	1.5.2	Parallel nuisance estimation
pyarrow	22.0.0	Parquet I/O for nuisance cache

Installing Python Dependencies

Installation (run once from the repository root):

```
pip install -r requirements.txt
```

requirements.txt:

```
numpy==2.0.0
pandas==2.2.2
scikit-learn==1.7.2
doubleml==0.11.1
statsmodels==0.14.6
matplotlib==3.9.0
scipy==1.16.2
joblib==1.5.2
pyarrow==22.0.0
```

To install without pinned versions (use if the above causes conflicts):

```
pip install numpy pandas scikit-learn doubleml statsmodels matplotlib scipy joblib
pyarrow
```

Key Configuration Parameters

Before running Stage 2, review these settings in `Python Code/Python DML Scripts/DML Identification.py`:

Parameter	Paper value	Notes
<code>tipping_point</code>	0 or 1	Use 0 for standard results and 1 for the balanced Figure 4 run
<code>N_REP</code>	30	Set to ≤ 5 for a faster verification run
<code>k_fold_vector</code>	[5]	Cross-fitting folds
<code>N_WORKERS</code>	6	Set to available CPU cores
<code>re_estimate_nuisance</code>	False	True forces cache rebuild
<code>HTE</code>	True	Loan $\times$ farm-size interaction
<code>HTE_GATE</code>	True	Farm-size quartile effects; disabled automatically when <code>tipping_point = 1</code>

`N_REP` and `k_fold_vector` must be set to the same values in `DML Benchmarking.py`. The paper was run on Windows 11 with 6 physical cores and 16 GB RAM.

References

Lars Vilhuber, Connolly, M., Koren, M., Llull, J., & Morrow, P. (2022). “A template README for social science replication packages (v1.1)”. *Zenodo*. <https://doi.org/10.5281/zenodo.7293838>

Evans School Policy Analysis and Research Group (EPAR). (2024). *LSMS Data Dissemination: Nigeria GHS*.

<https://github.com/EvansSchoolPolicyAnalysisAndResearch/LSMS-Data-Dissemination>